

Mock Exam 2 — Claude Certified Developer – Foundations (CCDV-F)

50 questions · 115 minutes · single best answer. Design and scoring rules: blueprint.md.

Take this cold — no notes, no repo, one pass, timed. Write your answers down, then score against Section C.

Section A — Exam Conditions

Property	This exam
Items	50
Time	115 minutes
Pace	~2.3 minutes per item
Borderline	72% raw (36/50)
Comfortable	80% raw (40/50)
Materials	Closed book — no notes, no repo, no docs, no model
Format	Single best answer, four options

Sit this one after all eight coaching sessions. It's the first honest full-blueprint measurement, so resist the urge to look anything up mid-exam — a question you had to check is a question you got wrong.

Domain	Questions	Question numbers
Applications and Integration	16	1, 4, 7, 10, 13, 16, 19, 22, 25, 28, 31, 34, 37, 40, 43, 49
Model Selection and Optimization	9	2, 8, 15, 20, 26, 32, 38, 44, 50
Agents and Workflows	7	3, 11, 18, 27, 35, 42, 46
Prompt and Context Engineering	6	5, 14, 23, 30, 39, 47
Tools and MCP	5	6, 17, 29, 41, 48
Security and Safety	4	9, 21, 33, 45
Claude Code	2	12, 36
Eval, Testing, and Debugging	1	24
Total	50	

Section B — Questions

Q1

A logistics customer describes the problem this way: "our dispatchers lose about twenty minutes a shift reading driver check-in messages one at a time — we want Claude to summarize them." That's the whole brief. Which set of questions derives the *infrastructure* requirements from it?

- **A.** Which summarization prompt scores best on a held-out sample, how many few-shot examples the output format needs, and whether XML tags should delimit each check-in
- **B.** Which model tier to start on, which `effort` level the summaries justify, which cache TTL the shared prefix should use, and how token cost per shift is tracked

- **C.** How fast a summary must return and measured from where dispatchers sit; how many arrive at peak; where the data may be processed and under which regulation; whose credentials the system acts under
- **D.** Whether dispatchers prefer bullets or prose, how long a summary should run, whether it should name the driver, and whether the tone should stay formal

Q2

An agent request is assembled from a system prompt, twelve prior conversation turns, six tool definitions, three tool results returned by the previous turn, and Claude's own reply. Which of these draw on the same context-window budget?

- **A.** Only the system prompt and the conversation history — tool definitions and tool results are metered separately
- **B.** All of them, including the tool definitions, the tool results, and the model's own output
- **C.** Everything except the model's output, which is billed but does not occupy the window
- **D.** Only content inside `messages`; `system` and `tools` sit outside the window

Q3

A team is building an internal research assistant and genuinely cannot tell yet whether the steps will turn out predictable enough to hard-code. What does Anthropic's guidance tell them to do?

- **A.** Build a workflow first, since predictability is always the safer default
- **B.** Build both and A/B them against live traffic
- **C.** Go straight to orchestrator-workers, which covers both the predictable and unpredictable cases
- **D.** Start with an agent, then extract deterministic workflow patterns as they emerge from real usage

Q4

Why does the operate phase of a Claude application never reach "done" the way it can for traditional software with a fixed spec?

- **A.** Model behavior can shift on a version bump and prompt effectiveness drifts as usage patterns evolve, so quality can move with no code change at all
- **B.** Anthropic requires deployed applications to be re-certified each quarter
- **C.** The Messages API is stateless, so conversation state has to be rebuilt on every request
- **D.** Eval suites are constructed during the operate phase, and an eval suite is never finished

Q5

A planning agent's context has grown to the point where the next call will not fit. Inspecting the transcript, the bulk of it is twelve turns of dialogue and intermediate reasoning about a design decision — the tool outputs were already small. The team needs to shrink the session without losing the decisions already made. What does compaction give here that pruning cannot, and what should be wired up alongside it?

- **A.** Compaction summarizes the non-recoverable dialogue and reasoning via an LLM call, keeping recent exchanges and key decisions — and a `PreCompact` hook can archive the full transcript first
- **B.** Pruning does the same job at lower cost, because the agent can re-fetch anything cleared by re-calling the tool that produced it
- **C.** Compaction is lossless and needs no model call, and a `PostToolUse` hook is the right place to archive the transcript before it is condensed
- **D.** Compaction raises the session's context-window ceiling for the rest of the run, so nothing is lost and no archiving step is involved

Q6

A developer registers two tools directly in the request's `tools` array and also connects an MCP server whose third tool arrives through the `ListToolsRequest` handshake. From Claude's perspective, how does the MCP-discovered tool differ from the two registered by hand?

- **A.** MCP tools are invoked through a separate endpoint and do not produce `tool_use` blocks
- **B.** MCP tools are exempt from the `tool_result` pairing rule, because the server returns results directly to the model
- **C.** It doesn't differ — same description-based routing, same message-block pairing rules; only who wrote and owns the definition changes
- **D.** MCP tools are always evaluated after locally registered ones, so a name collision resolves in favor of the local tool

Q7

An engineer needs Claude to read a 40-page contract that will be sent once, inline, in a single request. Their first attempt reuses the image-block shape they already had working:

```
{ "type": "image", "source": { "type": "base64", "media_type": "application/pdf", "data": "<bytes>" }, "name": "contract.pdf" }
```

Which corrected block matches how PDFs are actually supplied?

- **A.** Keep `type: "image"` and the `application/pdf` media type, but replace `name` with `title`
- **B.** Use `type: "document"` with a `source` of `{ "type": "text", "data": "<extracted-text>" }`, extracting the text yourself before the call
- **C.** Use `type: "document"` and supply the bytes under a required `name` field instead of a `source` object
- **D.** Switch the block to `type: "document"`, keep the same `base64` source with the `application/pdf` media type, and drop `name` in favor of the optional `title` field

Q8

A team ports a classification service from an older pinned model to Sonnet 5. They keep `temperature: 0` in the request body, which is what had been holding label casing stable. Every call now returns 400. What happened, and what is the fix?

- **A.** `temperature` must be sent as a string rather than a number on the newest models
- **B.** The newest models (Fable 5, Opus 5, Sonnet 5) do not accept non-default sampling parameters — `temperature`, `top_p`, and `top_k` all return 400 — so remove them and steer the output through prompting instead
- **C.** `temperature: 0` is the single unsupported value; any value above zero is accepted
- **D.** A 400 on a previously working request is the rate-limit signal on the newest models; back off and retry

Q9

An internal-only agent fetches supplier pages a user names and summarizes them. The team documented a decision not to validate fetched page content: "every user is an authenticated employee, the risk is the user, and we trust our own people." A summary is later written to a path nobody configured, traced to a line buried in one of the fetched pages. Why did the original reasoning not address prompt injection, and what is the fix?

- **A.** It did address it — an authenticated, internal-only user base removes the injection vector, so this must be a separate bug

- **B.** It failed only because employees can be phished; the fix is SSO plus MFA in front of the agent's entry point
- **C.** The hostile instruction arrived through content the agent retrieved on the user's behalf rather than through the user's own prompt — so the fix is two-sided: treat fetched content as data, and put a hook in front of the write tool that refuses actions triggered by untrusted input
- **D.** It failed because internal users can jailbreak the model too; the fix is a system-prompt line telling Claude to ignore any instruction found inside a fetched page

Q10

A monitoring pipeline attaches four dashboard screenshots to each request. Each screenshot is $1,400 \times 840$ pixels, within the tier's native resolution limits so no downscaling applies. Using the visual-token formula, roughly how many visual tokens do the four images consume per request?

- **A.** ~6,000
- **B.** ~1,500
- **C.** ~1,296
- **D.** ~42,000

Q11

A marketing team already scores generated copy against a written rubric they use in review, and they've confirmed that a second pass against that rubric measurably improves the draft. Which named workflow pattern fits?

- **A.** Prompt chaining
- **B.** Routing
- **C.** Parallelization by voting
- **D.** Evaluator-optimizer

Q12

An 847-line CLAUDE.md contains a correctly worded restriction against touching `/legacy/tokens/`. The agent edited a file there anyway. A grep confirms the line is present, unambiguous, and spelled correctly. What is the diagnosis, and the fix?

- **A.** The restriction needs to appear at both the top and the bottom of the file so it gets read twice
- **B.** Size is CLAUDE.md's main failure mode — a larger file makes any single instruction a smaller fraction of what loads, so 846 other lines diluted the rule; trim CLAUDE.md to constraints that actually change behavior and move historical or reference content elsewhere
- **C.** CLAUDE.md only loads when a file matching its `paths` frontmatter is touched, and no glob matched `/legacy/tokens/`
- **D.** CLAUDE.md is advisory in every permission mode, so the only durable fix is to run the session under `bypassPermissions` with deny rules instead

Q13

A team plans to upload a set of reference images once through the Files API and reference them by `file_id` on every subsequent request. The deployment target is Claude in Amazon Bedrock. What blocks the plan?

- **A.** The Files API is currently beta and is not available on Bedrock or Vertex AI — confirm availability for your deployment platform
- **B.** Bedrock accepts `file_id` references for PDFs only, not for images

- **C.** `file_id` references require the `mcp-client-2025-11-20` beta header, which this deployment does not set
- **D.** Nothing blocks it — the Files API behaves identically across all six deployment platforms

Q14

An extraction prompt was validated against 30 sample invoices and is clean on every one of them. In production it fails on invoices where the total amount appears twice — once as a subtotal partway down the page — picking the wrong one. The prompt names the exact output fields, the response parses as valid JSON every time, and the tone and scope are correct. Which technique is missing?

- **A.** A system prompt, or a more specific one
- **B.** Few-shot examples showing the structure Claude keeps inventing
- **C.** A constraint covering that variant — the prompt was validated against a narrow input set with no rule for the case that breaks it
- **D.** An output constraint naming the form, field names, and stopping point

Q15

A long-running summarizer starts returning replies that end mid-sentence. The response carries HTTP 200 and `stop_reason: "model_context_window_exceeded"`, with nothing in the error logs. A junior engineer proposes wrapping the call in a retry with exponential backoff. What actually happened, and why is the retry the wrong response?

- **A.** The request was rejected with a validation error before generation started because the input exceeded the window; a retry of the identical request fails identically
- **B.** The input fit, but generation itself hit the ceiling mid-response, so the model stopped and returned partial output instead of erroring — an identical retry reproduces it, and the application has to trim or summarize history before the call
- **C.** This is a rate limit surfacing under a different stop reason; honoring `retry-after` resolves it
- **D.** The output limit was reached, so raising `max_tokens` fixes it

Q16

At the API-mechanics level, which statement correctly separates extended thinking from adaptive thinking?

- **A.** Both are enabled with `thinking.type: "enabled"`; only the token accounting differs between them
- **B.** Adaptive thinking produces explicit `thinking` content blocks, while extended thinking is invisible to the caller
- **C.** Both mechanisms apply to every current model, and the choice is made per request
- **D.** Adaptive thinking (Fable 5, Opus 5, Sonnet 5) lets the model decide depth, tuned by `effort`; extended thinking (`thinking.type: "enabled"`, currently Haiku 4.5) produces explicit `thinking` content blocks — and on the newest models thinking content is omitted from the response by default

Q17

An MCP server backs an internal support console. Every turn should begin with the current on-call rota already in context; the rota lives at a single fixed address, and the team would rather not spend a tool round-trip fetching it. Which server primitive fits, and what caveat applies?

- **A.** A resource — read-only data fetched by address and placed directly into context with no tool call; note that support for resources varies by client, so verify before relying on it
- **B.** A tool annotated with `readOnlyHint`, since read-only tools execute concurrently
- **C.** A prompt, since the rota is fixed wording that should be maintained in one place on the server

- **D.** A tool with `defer_loading` turned off, so its definition loads upfront rather than on demand

Q18

In the multi-agent structure the material calls the most common and best-supported, what does the central component actually do?

- **A.** Executes each subtask itself and delegates only once it runs out of context
- **B.** Splits the work into a fixed number of equal shards decided before the request arrives
- **C.** Receives the request, maintains a list of worker agents, delegates tasks, and reviews and aggregates their output — typically without executing the work itself
- **D.** Operates as a peer among equals, with any agent free to take the coordination role on a given turn

Q19

What are the size limits on a single Message Batches API submission?

- **A.** 10,000 requests or 32MB, whichever limit hits first
- **B.** 100,000 requests, with no payload size limit
- **C.** 256 requests per batch, submitted up to 100,000 times per day
- **D.** 100,000 requests or 256MB, whichever limit hits first

Q20

A ticket-tagging feature runs zero-shot on Sonnet 5 and clears its eval bar. A cost review asks whether Haiku 4.5 could carry the same work. Run zero-shot on Haiku 4.5, the feature misses the bar: inconsistent label casing and one edge category tagged wrong. What is the reasonable next step?

- **A.** Stay on Sonnet 5 — a tier that misses the bar zero-shot cannot be brought up to it
- **B.** Add few-shot examples to the Haiku 4.5 prompt and re-run the eval: a more capable model often succeeds zero-shot where a smaller one needs examples, so examples can let a cheaper tier do the job — weighed against the tokens each example costs on every call
- **C.** Turn on adaptive thinking for Haiku 4.5 with a higher `effort` level, closing the capability gap without touching the prompt
- **D.** Send the examples once so the model retains them for later calls, since examples act as training data and then cost nothing per request

Q21

A tool call matches one deny rule, two allow rules, and an ask rule. What happens?

- **A.** It is blocked — precedence is deny > ask > allow, and a single deny blocks the action regardless of how many allow rules also match
- **B.** It prompts for confirmation, because ask outranks both deny and allow
- **C.** It is allowed, because two allow rules outweigh one deny
- **D.** The most specific rule wins regardless of its type, so the outcome depends on which rule targets the narrowest path

Q22

A European customer already runs Microsoft Foundry across their organization and asks to keep the Claude workload there. Their stated requirement is EU regional residency. The engineering lead reports back: "Foundry is on our approved platform list and the customer is an EU tenant, so residency is satisfied — we can proceed to build." What is wrong with that conclusion?

- **A.** Nothing — a platform's residency posture applies uniformly to every model it offers

- **B.** Foundry cannot serve EU residency requirements at all; EU residency requires Bedrock or Vertex AI
- **C.** Foundry satisfies the requirement only once the full model ID is pinned with the `anthropic.` prefix
- **D.** Foundry offers Claude in two hosting forms with different residency properties — "Hosted on Azure" (currently Opus 4.8, Sonnet 5, Haiku 4.5, inference end-to-end on Azure infrastructure) and "Hosted on Anthropic" (all other Foundry Claude models, inference on Anthropic-operated infrastructure and not sufficient for EU regional residency) — so residency has to be confirmed per model, not per platform

Q23

A developer has reworded a summarization prompt five times — stronger verbs, an "IMPORTANT:" prefix, sentences reordered, a closing reminder added. The output still comes back in the wrong shape, and the prompt is now three times its original length. Why has none of that worked?

- **A.** Each rewrite invalidated the cached prefix, so none of the changes were ever applied to the request
- **B.** Rewording changes how you say something; it does not add a missing structural technique — a prompt that gets longer with every iteration and still fails is the signal that diagnosis is being skipped and text is just being padded
- **C.** Emphasis markers like "IMPORTANT:" are ignored unless they appear in the system prompt rather than the user turn
- **D.** Five differing results is normal variance on a non-deterministic model; a sixth attempt with the same prompt will likely land

Q24

An eval suite has three kinds of case: inputs with exactly one correct label; inputs whose output is a JSON payload that must parse with all required fields present and in range; and inputs whose output is a free-text explanation judged for faithfulness and tone. Which grading assignment matches the output shapes?

- **A.** Exact/string match for the fixed labels, a code-graded check for the JSON payloads, LLM-as-judge for the free text
- **B.** LLM-as-judge for all three, since a judge subsumes what the cheaper methods can check
- **C.** Exact/string match for both the labels and the JSON payloads, LLM-as-judge for the free text
- **D.** Code-graded checks for all three, which keeps the whole suite cheap enough to run on every commit

Q25

An engineer lifts working first-party Claude API code into a Vertex AI deployment. The client posts the same JSON body — including `"model": "claude-sonnet-5"` — to the Vertex endpoint. The call fails with an error about a missing required field, and the model name in the body appears to be ignored entirely. What is the correct request shape?

- **A.** Keep the body as-is and add an `anthropic-version` request header; the model name belongs in the body on Vertex just as it does on the first-party API
- **B.** Move the model name into the body's `anthropic_version` field and drop `model` altogether
- **C.** Specify the model in the endpoint URL rather than the request body, and include `anthropic_version` as a required body field (for example `vertex-2023-10-16`)
- **D.** Prefix the model ID with `anthropic.` and leave the rest of the request unchanged

Q26

A Python service's p95 latency for a single Claude call is 4.2 seconds. A developer swaps the `Anthropic` client for `AsyncAnthropic` and awaits the call, expecting that number to drop. It doesn't move. Why, and what did the change actually buy?

- **A.** `AsyncAnthropic` only reduces latency when combined with streaming, which this service does not use
- **B.** The async client adds event-loop overhead, so per-request latency rises slightly and throughput stays flat
- **C.** Nothing — `AsyncAnthropic` is a deprecated alias for the synchronous client
- **D.** The request still returns in real time; async buys concurrency — the application can handle other work while a request is in flight — not lower per-request latency or lower cost

Q27

A customer wants Claude Code's agent loop, context management, and tool scaffolding without building any of it themselves, and would rather not stand up and secure an execution sandbox either. Their security policy is absolute on one point: the agent's code, filesystem, and network egress must never leave infrastructure they control. Which wiring path fits?

- **A.** Managed Agents — Anthropic runs both the loop and the sandbox, and the sandbox can be pinned to the customer's own region
- **B.** Self-hosted sandboxes — orchestration stays on Anthropic's side while tool execution moves into infrastructure the customer controls, so the agent's code, filesystem, and network egress never leave their environment
- **C.** A raw Messages API loop — the only configuration in which tool execution stays inside the customer's environment
- **D.** The Agent SDK with `setting_sources` left unset, which confines all filesystem access to the customer's process

Q28

Three weeks from a delivery deadline, a team chose the deployment platform they had shipped on twice before, because the migration path was familiar and the calendar was tight. Every functional test passed. At the customer's security review, the reviewer asked where data was processed. The chosen platform did not satisfy the customer's residency requirement; another platform the team had available, with regional deployment options, would have. The integration was rebuilt on the compliant platform. Which reading of this incident is correct?

- **A.** The failure was in the test suite — a residency assertion in the integration tests would have caught it before review
- **B.** The failure was the security review's timing; reviews of this kind belong before functional testing rather than after
- **C.** Familiarity answers whether the team can build quickly; it says nothing about whether the customer is allowed to run the result — for a regulated customer, compliance is usually pass/fail rather than a tradeoff, and the constraint belongs in scoping, where checking early costs a conversation instead of a rebuild
- **D.** The team should have measured latency from the customer's actual region, which is the measurement that would have surfaced the residency gap

Q29

A team commits a `.mcp.json` at the repo root registering a warehouse MCP server launched over stdio with `npx`. It works on the author's machine. Two teammates clone the repo and the server fails to start; a third teammate, who does Node work daily, has no trouble. The CI runner fails the same way the two teammates do. What is going on?

- **A.** `.mcp.json` is a local-scope file and is not read from the repo root, so each machine has to re-add the server entry itself
- **B.** The server has to use HTTP transport, because `.mcp.json` cannot register stdio servers

- **C.** A project-scope `.mcp.json` distributes the registration, but a stdio server still spawns a local subprocess on each machine — so every teammate and the CI runner needs the runtime installed, here Node for an `npm`-launched server
- **D.** The two teammates and the CI runner are missing the `SECRET` secret reference the server's header depends on, which is what prevents the subprocess from launching

Q30

A team switches an extraction endpoint over to JSON outputs with a fixed schema. They notice the first call after each deploy is noticeably slower than the calls that follow it. A second endpoint, which generates a different schema per tenant, is slow on nearly every call rather than just the first. What explains both observations?

- **A.** The schema is validated against the response after generation, and validation time grows with the number of distinct tenants
- **B.** Structured outputs disable prompt caching, so every request reprocesses the prefix from scratch
- **C.** The first call after a deploy pays a TLS handshake cost; the per-tenant endpoint is simply hitting rate limits
- **D.** The API compiles the schema into a grammar before it can constrain output, and compiled grammars are cached for 24 hours from last use — steady traffic on a stable schema pays the compile cost once, while a workload that changes schemas constantly pays it repeatedly

Q31

When is the decision about which content Claude may treat as instruction-bearing, versus which content it must treat as inert data, properly made?

- **A.** During incident response, once an injection attempt has been observed and its shape is known
- **B.** At design time, when the message flow is architected — your system prompt and your own user's direct input are instruction-bearing, while retrieved documents, tool results, and third-party content are inert data
- **C.** At request time by the model, which infers a trust level from the block type each piece of content arrives in
- **D.** At deploy time, by selecting the permission mode that matches the sensitivity of the content the agent will read

Q32

A compliance-summary workload runs on Haiku 4.5 with `thinking.type: "enabled"`, specifically so an auditor can read the reasoning blocks alongside each summary. A quality review moves the workload to Sonnet 5. The team lifts the request body across unchanged and expects identical behavior. What changes?

- **A.** Sonnet 5 uses adaptive thinking — the model decides when and how much to think, tuned via `effort` rather than the extended-thinking `thinking` configuration — and on the newest models thinking content is omitted from the response by default, so summarized display has to be requested explicitly
- **B.** Nothing changes; `thinking.type: "enabled"` is the shared configuration for both reasoning modes
- **C.** Sonnet 5 raises the thinking budget automatically, so the same configuration yields deeper but otherwise identical reasoning blocks
- **D.** Sonnet 5 has no reasoning mode at all; reasoning depth on Sonnet is governed only by `max_tokens`

Q33

During setup, an API key was pasted inline into `.mcp.json` and committed. Twenty minutes later the developer noticed, replaced the literal with a `${VAR}` reference, and pushed that as the next commit. What still has to happen, and why?

- **A.** Nothing further — the follow-up commit removed the literal value from the configuration the team actually loads
- **B.** Force-push a rewritten branch, which removes the key from everywhere it could have propagated
- **C.** Add a `PostToolUse` hook that scans future commits for credential-shaped patterns, which closes the exposure
- **D.** The key has to be rotated: overwriting the file in a later commit does not remove it from history, so anyone who ever had read access to the repo may have had the key — and the rotation has to account for every other service configured with the same value, since those consumers break when it changes

Q34

Which statement describes `CLAUDE.md`'s place in configuration management?

- **A.** It is generated by `/init` and should be treated as machine-managed output rather than hand-edited
- **B.** It holds user-scoped preferences, so it belongs in `~/.claude` and outside the repository
- **C.** It carries project and user conventions plus persistent context, and is version-controlled like code — one of four configuration artifacts (alongside `settings.json`, model version pins, and prompt versions) that get code-level rigor precisely because none of them are compiled or type-checked
- **D.** It is re-read only when its contents change, so version control adds nothing beyond a backup copy

Q35

An agent loop is running under the Agent SDK with a `PreToolUse` hook that denies any `write_file` whose path falls outside `/workspace/output`. On turn 6, Claude requests a write to `/etc/limits.conf`. Walk through what happens next.

- **A.** The hook's denial raises an exception inside the SDK, ending the session with an `error_during_execution` result subtype
- **B.** The tool never runs — the denial short-circuits the loop at that call, Claude receives the rejection as the tool result and typically tries a different approach, and because the hook runs in the application process rather than the model's context window it costs no tokens to enforce
- **C.** The write executes and the hook records it afterwards, since `PreToolUse` can inspect a call but not block it
- **D.** The call is placed in a confirmation queue and resumes if an operator approves it, since a hook denial maps to an `ask` decision

Q36

A team wants guidance about explicit SQL transaction boundaries to load only when Claude is working on files under `src/db/`, and to stay out of the way everywhere else. Where does that scoping come from?

- **A.** A `paths` glob in the rules file's YAML frontmatter (for example `paths: ["src/db/**/*.sql"]`) — not from the directory the file sits in; a rules file with no `paths` field loads unconditionally at the same priority as `CLAUDE.md`
- **B.** Placing the rules file inside `src/db/`, which scopes it to that subtree
- **C.** A `paths` entry in `.claude/settings.json` mapping directories to rules files
- **D.** A `disable-model-invocation: true` flag in the rules file's frontmatter, which limits it to the files under review

Q37

A "small wording tweak" to a production system prompt measurably degraded output quality. The prompt lives as a string literal that gets edited in place; nobody can say what it said last week, and the on-call engineer's only option is to rewrite it from memory and hope. Which practice was missing?

- **A.** Structured outputs on the endpoint, which would have held the output shape stable through the tweak
- **B.** Model version pinning, which would have prevented the behavior shift in the first place
- **C.** A `PreToolUse` hook validating the prompt before each request goes out
- **D.** Prompt versioning — system prompts and few-shot examples are production configuration, tracked in version control with changelogs and rollback capability, because a small wording change can measurably shift the output distribution and nothing else will catch the regression before users do

Q38

A configuration being migrated to a newer pinned model sets a large `budget_tokens` value to buy deep reasoning on a multi-step analysis task. What replaces it?

- **A.** `max_tokens`, raised to the same value, which is where reasoning tokens are now accounted for
- **B.** `thinking.type: "enabled"` alongside the existing budget, which is still honored behind a beta header
- **C.** An `effort` level — `budget_tokens` is deprecated and returns a 400 on the newest generations, and reasoning depth is set through `effort`, with `max` reserved for multi-step problems needing the deepest analysis
- **D.** A lower `temperature`, so the model spends more steps converging on an answer

Q39

An extraction endpoint currently prefills the assistant turn with `{"` to force the response into JSON. The team wants a hard guarantee instead of a nudge, so they add `output_config.format` with a `json_schema` while leaving the prefill in place. The request fails. What is the correct read?

- **A.** JSON outputs and message prefilling are mutually exclusive on the same request, so pick one — and since the schema does the constraining through constrained decoding, the prefill is what to drop
- **B.** The prefill has to be valid against the schema; `{"` is an incomplete fragment, so completing it to a full skeleton object resolves the conflict
- **C.** Prefilling works only with strict tool use, not JSON outputs, so the endpoint should be converted into a tool call
- **D.** The schema needs `additionalProperties: false` before a prefill will be accepted alongside it

Q40

A developer installs one plugin from the team's marketplace. Afterwards, the plugin list shows three installed plugins — two that nobody asked for. No `settings.json` file changed, and no other install command was run. What happened?

- **A.** The marketplace pushed its full catalog, because `extraKnownMarketplaces` auto-registers every plugin a marketplace lists
- **B.** The installed plugin declared dependencies in its `plugin.json`, and installing from a marketplace can auto-install those declared dependencies, with marketplace-entry fields able to override or supplement the declaration — a plugin's dependency graph is configuration you inherit alongside its headline capability
- **C.** Claude Code auto-installs any MCP server a plugin's `SKILL.md` refers to, and each server registers as its own plugin

- **D.** The two extra plugins were already installed but stayed hidden from the list until a marketplace source was added

Q41

An agent has two tools: `find_account(email)`, which returns an account ID, and `get_balance(account_id)`. Current models default to issuing multiple independent `tool_use` blocks in a single turn, and that is exactly what happens here — Claude emits both calls in one turn, with a plausible-looking but invented value in `account_id`, and the second call fails against a real account store. How should the exchange be structured?

- **A.** Keep both calls in the one turn and set `strict: true` on `get_balance`, so the invented argument is rejected before your code runs
- **B.** Keep both calls in the one turn, return `is_error: true` on the second `tool_result`, and let Claude retry the call within that same turn
- **C.** Merge the two tools into one behind a `type` parameter, which is the standard fix for calls that depend on each other
- **D.** Structure it as separate turns: the second call cannot be built correctly until the first result is back, so let the first `tool_use` be answered by its matching `tool_result` in the immediately following user turn and let Claude issue the second call after that — reaching for `disable_parallel_tool_use` if the workflow needs strictly one call per turn

Q42

The same support engineer works with an agent in many short sessions across a week, and each new session needs the decisions reached in the previous ones. Which memory scope fits?

- **A.** In-context — resend the whole accumulated history at the start of each new session
- **B.** Stateless — start each session fresh, which keeps per-session token cost flat and predictable
- **C.** External storage — state written to a database at session end and read back at session start, which is what the same-user-continuing-across-many-shorter-sessions case calls for
- **D.** In-context with compaction, since compaction is the only mechanism that carries state across a session boundary

Q43

A team's code-review checklist covers application logic. Prompt edits and tool-schema edits travel a "content-only" fast path with no reviewer attached, on the reasoning that neither is really code. After a one-word prompt edit shifted the output shape in production and broke a downstream parser, the team wants a durable fix. Which change addresses it?

- **A.** Bring prompt diffs and tool-schema diffs explicitly into code-review scope, and put the eval suite in CI so a prompt or model-version change fails the build the way a broken unit test does
- **B.** Keep the fast path but require a second engineer to run the changed prompt manually a few times before merging
- **C.** Add type annotations to the tool schema so the compiler catches breaking changes at build time
- **D.** Move prompts into the CLAUDE.md hierarchy so they are re-injected on every request rather than reviewed

Q44

A team marks a `cache_control` breakpoint after a 30,000-token block holding the system prompt and tool definitions, and chooses the 1-hour TTL because requests arrive in bursts about 40 minutes apart. Traffic volume is unchanged, but the input portion of the bill roughly doubles. Inspecting the block, its first line

reads `Current time: 2026-04-14T09:22:07Z`, regenerated on every request. What is happening, and what is the fix?

- **A.** The 1-hour TTL is billed for the full hour whether or not the cached prefix is ever read, so short bursts never recover the charge
- **B.** A single changed character before the cache point invalidates the cache and forces a fresh, paid write — so every request pays the write premium (roughly 2x base input at the 1-hour TTL) and none ever earns the read discount; caching only pays when reads outnumber writes, so move the volatile line after the breakpoint or out of the cached prefix entirely
- **C.** The block sits below the minimum token threshold, so nothing was ever cached; raise it above the per-block minimum
- **D.** Cache reads are billed at the same rate as base input tokens, so caching only ever buys latency — the doubled bill is the expected cost of the write premium

Q45

An EU customer's requirement is that model inference must never occur outside the EU. The team's proposal: stay on the first-party Claude API, place an EU-hosted proxy in front of it so all application traffic originates in the EU, and add a `region` field to the client configuration. Assess the proposal.

- **A.** It works — pinning a region in the client configuration is the accepted pattern here, and the EU-hosted proxy supplies the processing boundary
- **B.** It works, but only if the proxy strips personal data before forwarding each request
- **C.** It does not work: the direct Anthropic API does not currently provide EU data residency, and where your application calls from does not determine where inference runs — EU-residency workloads route through Amazon Bedrock or Google Vertex AI with the region pinned in the client configuration
- **D.** It does not work because EU residency requires a signed Business Associate Agreement, which the first-party API does not offer to EU customers

Q46

A team's priority for a Python service is validated, strongly typed inputs and outputs, with no need for multi-agent coordination machinery. Which of the frameworks the exam blueprint names fits that priority?

- **A.** LangGraph
- **B.** Strands Agents
- **C.** The Claude Agent SDK
- **D.** PydanticAI

Q47

A structured-output endpoint returns a payload that parses cleanly, carries every required field with values inside their declared ranges, and reads fluently. QA marks the feature green on that basis. What is the gap?

- **A.** Structure and meaning are separate validations — parsing, field presence, and range checks establish that the output is well-formed, not that it is right; meaning needs an eval with a model-graded judge, and a fluent, confident-sounding response is not evidence of correctness
- **B.** There is no gap — a schema-constrained response that parses is correct by construction, which is the point of constrained decoding
- **C.** The gap is `stop_reason`, but since the payload parsed it was neither a refusal nor a truncation, so nothing further is required
- **D.** The gap is caching — a stale cached prefix is the usual cause of a well-formed but wrong structured payload

Q48

What are the two halves of keeping an MCP server's credential out of a repository — one that communicates the intent and one that enforces it?

- **A.** A `.gitignore` entry for `.mcp.json` plus a pre-commit secret scanner
- **B.** A `${VAR}` reference in the config so the value lives in the environment or a secret store rather than the file, plus a `PreToolUse` hook that inspects writes and edits to `.mcp.json` for credential-shaped patterns and blocks them — the CLAUDE.md convention communicates intent, the hook enforces it regardless of what the model decides
- **C.** `bypassPermissions` disabled plus a `PostToolUse` hook that redacts the key once the write has completed
- **D.** OAuth on the server plus a rotation schedule, which removes the need for any config-level credential reference at all

Q49

An engagement build works and the deadline was met. The customer's repo path, two alerting thresholds, and three prompt fragments are hardcoded into the agent loop; the eval suite lives on one engineer's personal branch; and the two engineers who know which of those values are customer-specific versus load-bearing roll onto a new engagement next week. The team asks whether to invest in packaging now or wait until a reuse request actually arrives.

- **A.** Wait — packaging before a second engagement exists is speculative work, and the values can be parameterized once a concrete reuse case appears
- **B.** Package by writing a README that walks through the implementation, since that is what a future team will actually need to read
- **C.** Package now: separate the customer-specific values into documented, parameterized configuration with defaults and bundle the eval suite alongside the code, because the knowledge of what is customer-specific versus load-bearing is cheapest to write down before the people who hold it move on
- **D.** Package by committing the eval suite and leaving the hardcoded values as they are, since running the eval will show a future team which values matter

Q50

A cost model for a caching agent reads each response's usage fields and sums them into a single "total input tokens" figure. Why does that misstate cost, and what should it track instead?

- **A.** Cache reads, cache writes, and regular input tokens are priced differently, so one combined total misstates cost as soon as caching is in play — track the three separately, from the `usage` block on the raw Messages API or the `usage` and `total_cost_usd` fields on the Agent SDK's `ResultMessage`
- **B.** Output tokens are folded into the input total on cached requests, double-counting them
- **C.** Cached requests report zero input tokens, so the running total understates actual volume
- **D.** Input tokens are free once a cache breakpoint is set, so only output tokens need modeling

Section C — Answer Key and Explanations

Q1 — Answer: C

- **Why C is correct:** Infrastructure requirements are the non-functional constraints the deployment must satisfy, mostly not stated in the business problem but derived by asking what it implies: latency (how fast, measured where the user actually is), scale (how many requests, at what peak), residency (where data may be processed, under which regulation), and identity (who acts, under what credentials, what must be auditable). These four most often decide the deployment platform.
- **Why not A:** Prompt wording, example count, and delimiter choice are prompt-design decisions driven by the functional requirement, not infrastructure constraints.
- **Why not B:** Model tier, effort, and cache TTL are model-selection and cost-optimization choices made later, after the requirements are known.
- **Why not D:** Format, length, and tone are functional-requirement detail — what the system must do — not the constraints the deployment has to satisfy.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.requirements/infrastructure-dimensions`
- **Revise:** `2_applications_and_integration.md` → Understanding Requirements

Q2 — Answer: B

- **Why B is correct:** The context window is a fixed, shared budget: system prompt, full conversation history, injected documents, every tool definition, every tool result, and the model's own output all draw from the same pool.
- **Why not A:** Tool definitions and tool results are explicitly named as things that count against tokens — they are not metered separately.
- **Why not C:** The model's own output draws on the same window, which is why generation can hit the ceiling mid-response.
- **Why not D:** The `system` prompt and `tools` array are part of what the model processes and count against the same budget.
- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/context-budget`
- **Revise:** `4_model_selection_prompting_context.md` → LLM Fundamentals

Q3 — Answer: D

- **Why D is correct:** The decision rule is explicit about the uncertain case: if you're not sure, start with an agent and extract deterministic workflow patterns as they emerge from real usage, rather than guessing at a workflow structure up front.
- **Why not A:** Workflows are the right answer when you can enumerate the exact steps in code — not a universal default when you can't.
- **Why not B:** Building both doubles the work and still doesn't reveal the step structure before you commit to one.
- **Why not C:** Orchestrator-workers is a specific pattern for input-dependent decomposition, not a hedge for an undecided architecture.
- **Difficulty:** Easy
- **Domain:** Agents and Workflows

- **Tag:** `agents.architecture/workflow-vs-agent`
- **Revise:** `1_agents_and_workflows.md` → Agent Architecture

Q4 — Answer: A

- **Why A is correct:** Because model behavior can shift on a version bump and prompts drift in effectiveness as usage patterns evolve, the operate phase is never "done" the way it can be for traditional software with a fixed spec.
- **Why not B:** No quarterly re-certification requirement exists in the material.
- **Why not C:** Statelessness is a real property of the Messages API, but it's a per-request design fact, not the reason the operate phase never closes.
- **Why not D:** Eval suites belong to the build phase, not the operate phase — building them early is what makes iterate and operate tractable.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.lifecycle/operate`
- **Revise:** `2_applications_and_integration.md` → Systems Life Cycle

Q5 — Answer: A

- **Why A is correct:** When the bloat is dialogue and reasoning that can't be cheaply re-fetched, compaction summarizes older history via an LLM call while keeping recent exchanges and key decisions intact — and a `PreCompact` hook can archive the full transcript before it's summarized away.
- **Why not B:** Pruning is the lossless, cheap option for re-fetchable tool output. Dialogue and reasoning can't be re-fetched by re-calling a tool, so pruning would simply lose it.
- **Why not C:** Compaction costs an LLM call and is lossy by design; `PostToolUse` fires after a tool returns, not before compaction.
- **Why not D:** Compaction condenses what's inside the window; it does not change the window's size.
- **Difficulty:** Medium
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.context/compaction`
- **Revise:** `4_model_selection_prompting_context.md` → Context Engineering

Q6 — Answer: C

- **Why C is correct:** From Claude's perspective, a tool discovered via MCP's `ListToolsRequest` handshake is indistinguishable from one registered manually — same description-based routing, same message-block pairing rules. Only who wrote and owns the definition differs.
- **Why not A:** MCP tools produce ordinary `tool_use` blocks; what changes is that the harness dispatches them rather than your own in-process code.
- **Why not B:** The `tool_use` / `tool_result` pairing rules apply identically to MCP-sourced tools.
- **Why not D:** No ordering or collision-resolution rule of that kind exists in the material; routing is driven by descriptions.
- **Difficulty:** Medium
- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/discovery`
- **Revise:** `3_claude_code_tools_mcp.md` → MCP Server Development

Q7 — Answer: D

- **Why D is correct:** PDFs use a `document` block, not `image`, with the same source pattern (`base64`, `url`, or `file_id`), no required `name` field, and optional `title` and `context` fields.
- **Why not A:** The block type itself is wrong — swapping `name` for `title` on an `image` block doesn't make it a document block.
- **Why not B:** `source.type` is `base64`, `url`, or `file_id`; there's no text source type, and extracting the text yourself isn't the mechanism.
- **Why not C:** The bytes go in the `source` object, and `name` isn't a required field on a document block at all.
- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/documents`
- **Revise:** `2_applications_and_integration.md` → Claude API Mechanics (Vision)

Q8 — Answer: B

- **Why B is correct:** The newest Claude models (Fable 5, Opus 5, Sonnet 5) do not accept non-default sampling parameters — setting `temperature`, `top_p`, or `top_k` returns a 400 error, and output is steered through prompting instead.
- **Why not A:** Parameter type isn't the issue; the parameter isn't accepted at all on these models.
- **Why not C:** It isn't a value-range restriction — non-default sampling parameters are rejected, not specific values.
- **Why not D:** 429 is the rate-limit status. A 400 is a bad request, and it's explicitly non-retriable: the identical request fails identically.
- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/sampling`
- **Revise:** `4_model_selection_prompting_context.md` → LLM Fundamentals

Q9 — Answer: C

- **Why C is correct:** Trusting your own users does not solve prompt injection, because the hostile instruction typically arrives through content the agent retrieves rather than through the user's own prompt. The documented fix is two-sided: treat fetched content as data, and put a hook in front of the write tool that refuses actions triggered by untrusted input.
- **Why not A:** This is precisely the reasoning the incident pattern refutes — the injection came from a fetched page, not from a user.
- **Why not B:** Stronger user authentication doesn't touch the vector; the untrusted content is the page, not the user.
- **Why not D:** Instructing the model to treat fetched content as data helps but remains a soft boundary — the content can mimic delimiters or argue for an exception. The reliable boundary is what the agent is allowed to do as a result.
- **Difficulty:** Medium
- **Domain:** Security and Safety
- **Tag:** `sec.appsec/indirect-injection`
- **Revise:** `5_eval_debugging_security.md` → The mechanism behind prompt injection

Q10 — Answer: A

- **Why A is correct:** Claude views images in 28×28-pixel patches, so cost is `[width/28] × [height/28]`. Here that's $50 \times 30 = 1,500$ visual tokens per image, and 6,000 for four.

- **Why not B:** 1,500 is the cost of one screenshot; the request carries four.
- **Why not C:** ~1,296 is the worked example for a 1000×1000px image (36 × 36 patches), not these dimensions.
- **Why not D:** ~42,000 is what you get by dividing only one dimension by 28 (50 × 840) — the formula divides both.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/vision`
- **Revise:** `2_applications_and_integration.md` → Claude API Mechanics (Vision)

Q11 — Answer: D

- **Why D is correct:** Evaluator-optimizer has one LLM generate a response and a second evaluate it and give feedback for iterative refinement — the right fit for tasks with clear evaluation criteria where refinement demonstrably improves output, which is exactly what a written rubric and a confirmed second-pass gain describe.
- **Why not A:** Prompt chaining decomposes a task into sequential steps; there's one task here, revised.
- **Why not B:** Routing classifies input and sends it to a specialized handler; there are no distinct input categories.
- **Why not C:** Voting runs multiple attempts on the same task for confidence through diverse perspectives — it doesn't feed critique back for refinement.
- **Difficulty:** Easy
- **Domain:** Agents and Workflows
- **Tag:** `agents.patterns/evaluator-optimizer`
- **Revise:** `1_agents_and_workflows.md` → The five named workflow patterns

Q12 — Answer: B

- **Why B is correct:** This is the documented incident. Size is CLAUDE.md's main failure mode: a larger file makes any single instruction a smaller fraction of what loads, so an 847-line file diluted a correct path restriction. The fix is keeping CLAUDE.md to constraints that actually change behavior and moving historical or reference content elsewhere.
- **Why not A:** Duplication doesn't change the dilution ratio, and no such mechanism is described.
- **Why not C:** `paths`-based conditional loading belongs to rules instruction files. CLAUDE.md loads in full, every session, unconditionally.
- **Why not D:** `bypassPermissions` removes safety checkpoints rather than adding them, and it uniquely skips the protected-path guard other modes keep.
- **Difficulty:** Medium
- **Domain:** Claude Code
- **Tag:** `cc.operation/claude-md`
- **Revise:** `3_claude_code_tools_mcp.md` → CLAUDE.md, rules files, hooks, and subagents

Q13 — Answer: A

- **Why A is correct:** The `file` source type via the Files API is currently beta and is not available on Bedrock or Vertex AI — the material says to verify availability for your deployment platform explicitly.
- **Why not B:** No PDF-only restriction on `file_id` exists; the constraint is platform availability, not media type.
- **Why not C:** `mcp-client-2025-11-20` is the beta header for the API MCP connector's `mcp_toolset` configuration, unrelated to the Files API.

- **Why not D:** Feature availability differs by platform — the first-party API typically gets new features earliest, and this is one of the named gaps.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/files-api`
- **Revise:** `2_applications_and_integration.md` → Claude API Mechanics (Vision)

Q14 — Answer: C

- **Why C is correct:** The failure signature "clean on tested inputs, breaks on a variant or edge case" maps to a missing constraint covering that variant — the prompt was validated against a narrow input set with no rule for the case that breaks it (here: which of two candidate totals to take).
- **Why not A:** A missing or vague system prompt shows up as scope drift, tone shift, or degradation deeper into a conversation. Scope and tone are correct here.
- **Why not B:** Few-shot examples fix an invented structure — Claude doing the task in a shape you never specified. The shape is right; the field value is wrong.
- **Why not D:** A missing output constraint shows up as the wrong output *shape*. The output parses as valid JSON with the specified fields.
- **Difficulty:** Medium
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.prompting/constraints`
- **Revise:** `4_model_selection_prompting_context.md` → Prompt Engineering

Q15 — Answer: B

- **Why B is correct:** There are two distinct context-window failures. Input already larger than the window is rejected with a validation error before generation. Input that fits but hits the ceiling mid-response makes current models stop and return partial output with `model_context_window_exceeded` — not an error. The mitigation is trimming or summarizing history before each call, and the request isn't retrievable because the identical request reproduces it.
- **Why not A:** That describes the other failure mode; the response here has HTTP 200 and a stop reason, meaning generation started and ran.
- **Why not C:** A rate limit is a 429 with its own retrievable classification. This response is a successful call with a stop reason.
- **Why not D:** Hitting the output limit reports `stop_reason: "max_tokens"`. The stem states a different stop reason.
- **Difficulty:** Hard
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/context-window`
- **Revise:** `4_model_selection_prompting_context.md` → LLM Fundamentals

Q16 — Answer: D

- **Why D is correct:** These are two distinct reasoning mechanisms that don't both apply to the same model. Adaptive thinking (Fable 5, Opus 5, Sonnet 5) has the model decide depth, tuned by `effort`; extended thinking (`thinking.type: "enabled"`, currently Haiku 4.5) produces explicit `thinking` content blocks. Thinking content is omitted from the response by default on the newest models, so summarized display has to be requested.
- **Why not A:** `thinking.type: "enabled"` is the extended-thinking configuration; adaptive thinking is tuned via `effort` instead.

- **Why not B:** It's the reverse — extended thinking is what produces the explicit `thinking` blocks.
- **Why not C:** They don't both apply to the same model, which is the whole point of checking the per-model capability table.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/thinking`
- **Revise:** `2_applications_and_integration.md` → Claude API Mechanics (Extended thinking and adaptive thinking)

Q17 — Answer: A

- **Why A is correct:** Resources are read-only data fetched by address and placed directly into context with no tool call — the right primitive when known data should be in context from the start of a turn and pulling it in directly is cheaper and more predictable than a tool round-trip. The stated caveat is that support varies by client, so verify before relying on it.
- **Why not B:** A tool means a tool call and a round-trip, which is exactly what the team wants to avoid; `readOnlyHint` governs concurrent execution, not context placement.
- **Why not C:** Prompts are server-exposed instruction templates invoked by name — the primitive for when specific wording matters, not for getting data into context.
- **Why not D:** `defer_loading` controls when a tool *definition* loads to save context; it doesn't place the rota data into context.
- **Difficulty:** Easy
- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/primitives`
- **Revise:** `3_claude_code_tools_mcp.md` → MCP Server Development (Three server capabilities)

Q18 — Answer: C

- **Why C is correct:** The supervisor (leader-worker) pattern is described as the most common and best-supported multi-agent structure: a central supervisor receives the request, maintains a list of worker agents, delegates tasks, and reviews and aggregates their output — typically without executing the work directly.
- **Why not A:** The supervisor's defining property is that it delegates rather than executing; context exhaustion isn't the trigger.
- **Why not B:** Fixed pre-decided sharding describes neither the supervisor pattern nor orchestrator-workers, whose decomposition is dynamic.
- **Why not D:** A rotating peer structure isn't among the named structures; the hierarchical variant instead adds mid-level supervisors between a top-level manager and leaf workers.
- **Difficulty:** Easy
- **Domain:** Agents and Workflows
- **Tag:** `agents.architecture/supervisor`
- **Revise:** `1_agents_and_workflows.md` → Manager/supervisor hierarchies

Q19 — Answer: D

- **Why D is correct:** A single batch call accepts up to 100,000 requests or 256MB, whichever limit hits first. You submit once, get a `batch_id` back, poll for completion, and download the results.
- **Why not A:** Both numbers are wrong; neither figure appears in the batch mechanics.
- **Why not B:** There is a payload size limit, and 256MB can bind before the request count does.

- **Why not C:** 256 is the megabyte payload ceiling, not a per-batch request count, and no daily submission cap of that shape is described.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/batch`
- **Revise:** `2_applications_and_integration.md` → Realtime vs. batch (Batch API mechanics)

Q20 — Answer: B

- **Why B is correct:** Examples live in the prompt and show the model the exact shape of an answer that a description can't pin down. A more capable model often succeeds zero-shot where a smaller model needs a few examples, so adding examples can let a cheaper model do the job — and since each example costs tokens on every call, it's a quality/cost tradeoff resolved by re-running the eval.
- **Why not A:** The material's explicit position is the opposite: adding examples is how a cheaper tier can be brought up to the bar.
- **Why not C:** Haiku 4.5 supports extended thinking, not adaptive thinking — `effort` is the control for adaptive thinking on Fable 5, Opus 5, and Sonnet 5.
- **Why not D:** Examples aren't training data; they live in the prompt and are re-sent, which is precisely why they cost tokens on every call.
- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/shot-count`
- **Revise:** `4_model_selection_prompting_context.md` → LLM Fundamentals (Prompting modes)

Q21 — Answer: A

- **Why A is correct:** When multiple hooks or rules apply to the same action, precedence is deny > ask > allow — a single deny blocks the action regardless of how many allow rules also match. That ordering is what makes a hook a real boundary rather than a best-effort check.
- **Why not B:** Ask sits between deny and allow, so a deny still wins over it.
- **Why not C:** Rules don't vote; deny is absolute.
- **Why not D:** Precedence is by rule type, not by specificity of the path a rule targets.
- **Difficulty:** Easy
- **Domain:** Security and Safety
- **Tag:** `sec.hooks/precedence`
- **Revise:** `5_eval_debugging_security.md` → Hooks as enforcement, not convention

Q22 — Answer: D

- **Why D is correct:** Microsoft Foundry offers Claude in two hosting forms with different residency properties: "Hosted on Azure" (currently Opus 4.8, Sonnet 5, Haiku 4.5, inference end-to-end on Azure infrastructure) and "Hosted on Anthropic" (all other Foundry Claude models, inference on Anthropic-operated infrastructure, not sufficient for EU regional residency). Residency has to be confirmed per model, not per platform — the platform name alone doesn't tell you where a given model's inference runs.
- **Why not A:** That's the exact assumption the gotcha exists to break: one platform, two residency postures depending on the model.
- **Why not B:** Foundry's "Hosted on Azure" form does run inference end-to-end on Azure infrastructure, so Foundry isn't categorically ruled out.
- **Why not C:** The `anthropic.` prefix is Bedrock's full-model-ID format, and version pinning is a change-control concern, not a residency control.

- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/platforms`
- **Revise:** `2_applications_and_integration.md` → Deployment platforms (Microsoft Foundry gotcha)

Q23 — Answer: B

- **Why B is correct:** Rewording changes how you say something; it doesn't add a missing technique. A prompt that keeps getting longer with every iteration and still fails is the signal that diagnosis is being skipped and text is just being padded — the instruction is to stop and identify which structural piece (output constraint, system prompt, few-shot examples, or a constraint covering the failing variant) is actually absent.
- **Why not A:** Cache invalidation affects cost and latency, not whether a changed prompt takes effect.
- **Why not C:** No such rule exists; the system prompt carries the whole-session behavioral contract, but emphasis markers aren't conditionally ignored in the user turn.
- **Why not D:** A consistently wrong output *shape* isn't sampling variance — it's a structural gap that another attempt at the same prompt won't close.
- **Difficulty:** Medium
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.prompting/diagnosis`
- **Revise:** `4_model_selection_prompting_context.md` → Prompt Engineering

Q24 — Answer: A

- **Why A is correct:** The grading method follows the output shape: exact/string match for one correct label or value, a code-graded check for structured or code output (parses, valid syntax, in range, required field present), and LLM-as-judge for open-ended quality like faithfulness, instruction-following, and tone.
- **Why not B:** A judge is a second model call per case — noisy, costly, and meaningless until calibrated. Grade format and structure with code and reserve the judge for what only a judge can assess.
- **Why not C:** Exact match on a JSON payload fails any valid reordering or paraphrase; a code-graded check is what tests structure without over-constraining text.
- **Why not D:** Code-graded checks say nothing about whether the content is good, only that it's well-formed — they can't assess faithfulness or tone.
- **Difficulty:** Easy
- **Domain:** Eval, Testing, and Debugging
- **Tag:** `eval.debugging/grading`
- **Revise:** `5_eval_debugging_security.md` → Matching the grading method to the output shape

Q25 — Answer: C

- **Why C is correct:** Google Cloud's API shape differs from the first-party API in exactly these two ways: the model is specified in the endpoint URL rather than the request body, and `anthropic_version` is a required body field (for example `vertex-2023-10-16`). That explains both symptoms — the ignored body `model` and the missing-required-field error.
- **Why not A:** `anthropic_version` is a required body field on Vertex, not a request header, and the model does not belong in the body there.
- **Why not B:** `anthropic_version` carries a version string like `vertex-2023-10-16`, not a model name.
- **Why not D:** The `anthropic.`-prefixed full model ID is Bedrock's identifier format and doesn't address Vertex's model-in-URL requirement or the missing body field.
- **Difficulty:** Medium

- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/vertex`
- **Revise:** `2_applications_and_integration.md` → Deployment platforms (Google Vertex AI)

Q26 — Answer: D

- **Why D is correct:** The Python SDK's `AsyncAnthropic` exposes non-blocking `async` / `await` calls that don't tie up the application thread, but the request still returns in real time — `async` buys concurrency, not lower per-request latency or lower cost.
- **Why not A:** Streaming changes when output becomes visible, not the `async` client's latency behavior; combining them doesn't shorten a single call.
- **Why not B:** No overhead-penalty claim of this kind appears in the material, and concurrency is exactly what the `async` client does add.
- **Why not C:** `AsyncAnthropic` is the Python SDK's current `async` client. In the TypeScript SDK there's no separate `async` class because its standard client is already Promise-based.
- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.technical/async`
- **Revise:** `4_model_selection_prompting_context.md` → Technical Fundamentals

Q27 — Answer: B

- **Why B is correct:** Self-hosted sandboxes are the named middle ground: orchestration stays on Anthropic's side while tool execution moves into infrastructure you control, so the agent's code, filesystem, and network egress never leave your environment — and you still avoid running the orchestration loop yourself.
- **Why not A:** With Managed Agents, Anthropic runs both the loop and the sandbox, and sessions are stateful and stored server-side — which is the opposite of the customer's constraint. No region-pinning of the managed sandbox is described.
- **Why not C:** A raw loop keeps execution in your environment, but so does the Agent SDK — and the customer explicitly doesn't want to build the loop.
- **Why not D:** `setting_sources` governs whether filesystem sources such as `CLAUDE.md`, skills, and hooks are loaded; leaving it unset is the classic surprise that keeps skills from loading, not a security boundary.
- **Difficulty:** Medium
- **Domain:** Agents and Workflows
- **Tag:** `agents.construction/deployment`
- **Revise:** `1_agents_and_workflows.md` → Managed agent deployment models

Q28 — Answer: C

- **Why C is correct:** This is the documented incident pattern, and its stated lesson: familiarity answers whether the team can build quickly; it says nothing about whether the customer is allowed to run the result. For a regulated customer, compliance is usually pass/fail rather than a tradeoff, and it belongs in scoping — checking early costs a conversation, checking late costs an entire rebuild.
- **Why not A:** Residency is determined by the platform, not by your code, so no assertion inside the integration tests could establish it. The build passed every functional test.
- **Why not B:** The review found a real, pre-existing mismatch. Moving the review earlier in the build phase still leaves the constraint discovered after platform selection; the requirements phase is where it belonged.

- **Why not D:** Latency and compliance are separate comparison dimensions. Measuring round-trip time from the customer's region tells you nothing about where data is processed.
- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.design/platform-choice`
- **Revise:** `2_applications_and_integration.md` → Comparing platforms so the choice survives a review

Q29 — Answer: C

- **Why C is correct:** A committed `.mcp.json` at the repo root is project scope — the whole team gets the registration automatically on clone. But for a stdio server that still spawns a local subprocess per teammate, so each machine needs the runtime installed (Node, for an `npm`-launched server). The one teammate who does Node work daily already has it; the others and the CI runner don't.
- **Why not A:** Local scope lives in `~/.claude.json` as a per-project entry. `.mcp.json` committed at the repo root is exactly how project scope is distributed.
- **Why not B:** `.mcp.json` registers stdio servers fine. HTTP is recommended for anything not local, but that's a transport-choice recommendation, not a config-file limitation.
- **Why not D:** A missing `${VAR}` reference would fail uniformly, including for the Node-fluent teammate. The split along who has the runtime is what points at the subprocess requirement.
- **Difficulty:** Hard
- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/transport-scope`
- **Revise:** `3_claude_code_tools_mcp.md` → MCP Server Development (Transport, Configuration scope)

Q30 — Answer: D

- **Why D is correct:** Structured outputs work by constrained decoding, and the API has to compile your schema into a grammar before it can constrain output. Compiled grammars are cached for 24 hours from last use, so steady traffic on a stable schema pays the compile cost once while a workload that changes schemas constantly pays it repeatedly.
- **Why not A:** Constrained decoding enforces the schema token by token during generation, not by validating afterwards.
- **Why not B:** No such incompatibility is described. Structured outputs do add a small token cost (the API injects a system prompt describing the format), but that isn't a first-call latency spike.
- **Why not C:** A handshake wouldn't explain the per-tenant endpoint being slow on nearly every call, and rate limits surface as 429s rather than as a uniform latency increase.
- **Difficulty:** Hard
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.output/structured-outputs`
- **Revise:** `4_model_selection_prompting_context.md` → Output Handling

Q31 — Answer: B

- **Why B is correct:** Deciding what content Claude may treat as instruction-bearing (your system prompt, your own user's direct input) versus what it must treat as inert data (retrieved documents, tool results, third-party content) is an application-design decision made when you architect the message flow — not a patch applied after an incident.
- **Why not A:** Making it an incident-response decision is the failure the material is warning against; the boundary is a design-time choice.

- **Why not C:** The model reads its entire context as one undifferentiated stream of tokens with no structural marker separating trusted from untrusted content — it can't infer a trust level for you.
- **Why not D:** Permission modes bound what the agent may *do*. They don't decide which content counts as instruction-bearing.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.design/content-boundaries`
- **Revise:** `2_applications_and_integration.md` → Claude Application Design (Content boundaries)

Q32 — Answer: A

- **Why A is correct:** Sonnet 5 uses adaptive thinking — the model decides when and how much to think, tuned via `effort` rather than a fixed token budget or the extended-thinking `thinking` configuration. On the newest models thinking content is omitted from the response by default, so if reasoning has to be shown to a user or auditor, summarized display must be requested explicitly.
- **Why not B:** `thinking.type: "enabled"` is extended thinking's configuration, which is the Haiku 4.5 mechanism here — the two reasoning modes are not the same feature and don't share configuration.
- **Why not C:** There's no automatic budget escalation; the older `budget_tokens` control is deprecated and returns a 400 on the newest generations.
- **Why not D:** Sonnet 5 does have a reasoning mode — adaptive thinking. `max_tokens` bounds output length, not reasoning depth.
- **Difficulty:** Hard
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.model-choice/reasoning-modes`
- **Revise:** `4_model_selection_prompting_context.md` → Model Selection and Tradeoffs

Q33 — Answer: D

- **Why D is correct:** A secret in committed configuration is a permanent exposure: it enters repository history, and overwriting the file in a later commit does not remove it from history. Anyone who ever had read access may have had it, so the only correct response is rotation — and in the documented incident the rotation broke two other services configured with the same key, which is why you keep a record of which services consume each credential.
- **Why not A:** The team's current config is clean, but the key still exists in history, in clones, and in any CI runner that pulled it.
- **Why not B:** Rewriting history doesn't reach clones a teammate already has or a CI runner that already read the value, and it isn't the prescribed response — rotation is.
- **Why not C:** A scanner or a `PreToolUse` hook prevents the *next* leak. Neither un-exposes a key that has already been committed.
- **Difficulty:** Medium
- **Domain:** Security and Safety
- **Tag:** `sec.secrets/rotation`
- **Revise:** `5_eval_debugging_security.md` → Identity, Secrets, and Key Management

Q34 — Answer: C

- **Why C is correct:** `CLAUDE.md` holds project and user conventions plus persistent context and is version-controlled like code. It's one of four configuration artifacts — with `settings.json`, model version pinning, and prompt versioning — that all get code-level rigor precisely because none of them are compiled or type-checked, so nothing else will catch a regression before users do.

- **Why not A:** `/init` generates a starting version from the codebase, but the instruction is to validate it before trusting it — it's a hand-maintained artifact.
- **Why not B:** There's a CLAUDE.md hierarchy with a project scope; the project file is a committed, team-wide artifact, not just a personal preference.
- **Why not D:** CLAUDE.md loads in full, every session, unconditionally. Version control buys review, changelogs, and rollback, not just a backup.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.config/claude-md`
- **Revise:** `2_applications_and_integration.md` → Configuration Management

Q35 — Answer: B

- **Why B is correct:** A `PreToolUse` hook that rejects a call short-circuits the loop entirely — the tool never runs, Claude receives the rejection as the tool result and typically tries a different approach. Hooks run in your application process rather than inside the model's context window, so they consume no tokens.
- **Why not A:** Hitting a hook denial isn't an execution error. The `error_during_execution` subtype is a loop-level result state, and a denial instead feeds a rejection back into the loop.
- **Why not C:** `PreToolUse` fires before a tool executes and can deny the call outright or modify its input. `PostToolUse` is the one that can't block.
- **Why not D:** A denial is a deny decision, not an ask — and deny takes precedence over ask and allow, so there's nothing to confirm.
- **Difficulty:** Hard
- **Domain:** Agents and Workflows
- **Tag:** `agents.construction/hooks`
- **Revise:** `1_agents_and_workflows.md` → Hooks for deterministic actions

Q36 — Answer: A

- **Why A is correct:** Rules instruction files in `.claude/rules/` load only when Claude works with matching files, via a `paths` glob in YAML frontmatter. The scoping comes from the frontmatter, not from directory placement — and a rules file with no `paths` field loads unconditionally at the same priority as CLAUDE.md, regardless of which subdirectory it sits in.
- **Why not B:** Directory placement has no scoping effect; that's the specific misconception this mechanism trips people on.
- **Why not C:** `settings.json` carries tool permissions, hooks, and MCP server registration — not directory-to-rules-file mappings.
- **Why not D:** `disable-model-invocation: true` belongs to a skill's frontmatter and makes a workflow explicitly-invoked-only; it isn't a path-scoping mechanism.
- **Difficulty:** Easy
- **Domain:** Claude Code
- **Tag:** `cc.operation/rules-files`
- **Revise:** `3_claude_code_tools_mcp.md` → CLAUDE.md, rules files, hooks, and subagents

Q37 — Answer: D

- **Why D is correct:** System prompts and few-shot examples are production configuration, not throwaway text — track them the way you track code, with version control, changelogs, and rollback capability, because a small wording tweak can measurably shift the output distribution and nothing else (no

compiler, no type system) will catch the regression before users do. The missing rollback capability is exactly what left the on-call engineer rewriting from memory.

- **Why not A:** Structured outputs would hold the response *shape*; the degradation here is in quality, and the missing artifact is the prior prompt version.
- **Why not B:** Nothing in the scenario indicates a model change — a human edited the prompt. Pinning addresses version bumps, not prompt edits.
- **Why not C:** Hooks gate tool calls in the agent loop; they don't version a prompt or provide a rollback path.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.config/prompt-versioning`
- **Revise:** `2_applications_and_integration.md` → Configuration Management

Q38 — Answer: C

- **Why C is correct:** `budget_tokens` is deprecated and, on the newest model generations, returns a 400 error. Reasoning depth on adaptive thinking is set through `effort` (`low`, `medium`, `high`, `xhigh`, `max`), with `max` described as maximum depth for multi-step problems needing deep analysis.
- **Why not A:** `max_tokens` bounds the response length. It isn't a reasoning-depth control.
- **Why not B:** `thinking.type: "enabled"` is extended thinking's configuration, and no beta header revives `budget_tokens`.
- **Why not D:** Sampling parameters don't govern reasoning depth — and on the newest models non-default `temperature` returns a 400 anyway.
- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/effort`
- **Revise:** `4_model_selection_prompting_context.md` → Model Selection and Tradeoffs (reasoning mode)

Q39 — Answer: A

- **Why A is correct:** JSON outputs and message prefilling are mutually exclusive on the same request — pick whichever pattern fits the task. Since the schema enforces validity on every token through constrained decoding, the prefill is redundant and is the piece to remove.
- **Why not B:** The incompatibility isn't about the prefill's content; completing the fragment doesn't make the two mechanisms combinable.
- **Why not C:** Strict tool use constrains the arguments Claude passes to your tools, and no prefill-plus-strict-tool-use exception is described.
- **Why not D:** `additionalProperties` shapes what the schema permits; it has nothing to do with the prefill restriction.
- **Difficulty:** Hard
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.output/prefilling`
- **Revise:** `4_model_selection_prompting_context.md` → Output Handling

Q40 — Answer: B

- **Why B is correct:** Installing from a marketplace can auto-install a plugin's declared dependencies — dependency resolution is declared in `plugin.json`, with marketplace-entry fields able to override or supplement it. A plugin's own dependency graph is configuration you inherit, not just its headline capability.

- **Why not A:** A managed marketplace allowlist restricts which marketplace sources users may add rather than auto-registering plugins, and `extraKnownMarketplaces` pushes a marketplace to users — neither installs a catalog.
- **Why not C:** No mechanism auto-installs MCP servers referenced from a skill body, and MCP servers aren't plugins in their own right.
- **Why not D:** Nothing describes installed-but-hidden plugins surfacing when a marketplace source is added.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.design/plugin-dependencies`
- **Revise:** `2_applications_and_integration.md` → Claude Application Design (Plugin management)

Q41 — Answer: D

- **Why D is correct:** Current models default to issuing multiple independent `tool_use` blocks in one turn when subtasks don't depend on each other. When one tool's output feeds the next call's input, structure it as separate turns — the second call can't be correctly built until the first result is back. A `tool_use` block must be answered by a matching `tool_result` in the immediately following user turn, and `disable_parallel_tool_use` forces strictly one call per turn where a workflow requires it.
- **Why not A:** Strict tool use constrains arguments against the tool's input schema. An invented account ID that is still a well-formed string satisfies the schema, so it passes validation and fails downstream anyway.
- **Why not B:** Setting `is_error: true` is the correct way to surface a tool failure to Claude, but it doesn't fix the ordering problem — and a retry arrives as a new turn, not inside the current one.
- **Why not C:** Merging behind a `type` parameter is the fix for two tools whose descriptions can't be cleanly separated even with exclusion conditions. It's the wrong instrument for a data dependency between calls.
- **Difficulty:** Hard
- **Domain:** Tools and MCP
- **Tag:** `tools.implementation/turn-structure`
- **Revise:** `3_claude_code_tools_mcp.md` → Tool Implementation (message block structure, subtask dependency)

Q42 — Answer: C

- **Why C is correct:** External storage — state written to a database at session end and read back at session start — is described as right for the same user or task continuing across many separate, shorter sessions over time. That is exactly this shape.
- **Why not A:** This is the documented failure. In-context memory suits single continuous sessions whose whole history fits comfortably; resending accumulated history at the start of many short sessions is what pushed injected history past 40K tokens before the first tool call in the incident pattern.
- **Why not B:** Stateless suits fully independent jobs, like a formatter that transforms one file and terminates. Here each session needs prior decisions.
- **Why not D:** Compaction manages the live context window inside a session; it isn't a cross-session persistence mechanism.
- **Difficulty:** Easy
- **Domain:** Agents and Workflows
- **Tag:** `agents.patterns/memory`
- **Revise:** `1_agents_and_workflows.md` → Agent memory: three scopes

Q43 — Answer: A

- **Why A is correct:** For Claude-application code, review coverage should explicitly include prompt and tool-schema changes, not just the surrounding application logic — a schema change is a breaking-change risk for every caller of that tool. And eval suites belong in CI the same way unit tests do, so a prompt or model-version change fails a build rather than surfacing as a silent quality regression in production.
- **Why not B:** Manual spot-checking a non-deterministic system is the practice evals exist to replace; a few runs by a second engineer isn't a repeatable gate.
- **Why not C:** There is no compiler for a tool schema or a prompt. That absence is the whole reason review and eval discipline have to cover them.
- **Why not D:** CLAUDE.md governs Claude Code sessions and doesn't apply to raw API calls at all; re-injection isn't a substitute for review.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.swe/code-review`
- **Revise:** `2_applications_and_integration.md` → Software Engineering Foundations

Q44 — Answer: B

- **Why B is correct:** A single changed character before the cache point invalidates the cache and forces a fresh, paid write. With a regenerated timestamp at the top of the block, every request is a write and none is a read, so the workload pays the write premium (roughly 2x base input at the 1-hour TTL) every time and never earns the read discount. The economics only work when reads outnumber writes, so the volatile line has to move after the breakpoint or out of the cached prefix.
- **Why not A:** The TTL governs how long a cached prefix survives, not a standing hourly charge; writes and reads are what get billed.
- **Why not C:** Caching requires a minimum token threshold per block, but 30,000 tokens is far above it — that isn't the constraint here.
- **Why not D:** Cache reads cost a fraction of standard input (roughly 0.1x), which is exactly why caching saves money when reads happen at all.
- **Difficulty:** Hard
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.cost/caching`
- **Revise:** `4_model_selection_prompting_context.md` → Cost and Token Management

Q45 — Answer: C

- **Why C is correct:** The direct Anthropic API does not currently provide EU data residency. Residency is about where model execution happens, which the location of your calling code doesn't change — EU-residency partners route through Amazon Bedrock or Google Vertex AI with the region pinned in the client configuration.
- **Why not A:** Pinning a region in the client config is the right pattern, but only on a cloud-mediated route that actually offers EU regional execution. A `region` field pointed at the first-party API doesn't create residency, and defaulting to a global endpoint with no explicit region is the common failure.
- **Why not B:** Stripping personal data is a separate data-minimization decision and doesn't establish where inference occurs.
- **Why not D:** A Business Associate Agreement is the HIPAA instrument for PHI. It isn't what GDPR residency turns on.
- **Difficulty:** Hard
- **Domain:** Security and Safety

- **Tag:** `sec.guardrails/residency`
- **Revise:** `5_eval_debugging_security.md` → Regulated data constraints

Q46 — Answer: D

- **Why D is correct:** PydanticAI is described as a Python-first framework built around type safety and schema validation without heavy orchestration machinery — the strongest choice when the priority is validated, typed inputs and outputs rather than multi-agent coordination.
- **Why not A:** LangGraph's defining characteristic is graph-based orchestration with explicit nodes and transitions — more upfront structure, aimed at complex stateful workflows.
- **Why not B:** Strands Agents is AWS's open-source, model-driven framework, deeply integrated with Amazon Bedrock — the natural pick inside an AWS-centric stack.
- **Why not C:** The Agent SDK is the provider-native primitive for the Claude agent loop; you reach for a cross-provider framework when you need patterns like strict typing that the SDK doesn't supply out of the box.
- **Difficulty:** Easy
- **Domain:** Agents and Workflows
- **Tag:** `agents.patterns/frameworks`
- **Revise:** `1_agents_and_workflows.md` → Third-party agentic frameworks

Q47 — Answer: A

- **Why A is correct:** Validate structure (does it parse, are required fields present, are values in range) separately from validating meaning, which needs an eval with a model-graded judge rather than a unit-test assertion. A fluent, confident-sounding response is not evidence of correctness — a guaranteed schema is not a guaranteed success.
- **Why not B:** Constrained decoding guarantees the output is valid against the schema, not that the extracted values are the right ones. Shape-correctness and value-correctness are different guarantees.
- **Why not C:** Checking `stop_reason` matters because a refusal or a truncation can still break a "guaranteed" schema — but clearing that check only confirms the response is complete, not that its content is right.
- **Why not D:** A stale cached prefix is a real caching tradeoff, but nothing here points at caching, and it isn't the general reason a well-formed payload can be wrong.
- **Difficulty:** Medium
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.output/validation`
- **Revise:** `4_model_selection_prompting_context.md` → Output Handling

Q48 — Answer: B

- **Why B is correct:** These are the two halves named in the incident fix: reference `${VAR}` in the config so the credential never travels with the file that references it (the value lives in the environment or a secret store), and back the CLAUDE.md convention with a `PreToolUse` hook that inspects writes and edits to `.mcp.json` for credential-shaped patterns and blocks them. The instruction communicates intent; the hook enforces it regardless of what the model decides.
- **Why not A:** `.mcp.json` at project scope is meant to be committed so the whole team gets the registration — gitignoring it defeats the point, and a scanner alone doesn't stop the write.
- **Why not C:** `PostToolUse` fires after a tool returns and cannot block, so redaction after the fact means the value was already written.

- **Why not D:** OAuth is the pattern for remote servers where the user's identity is part of authorization; a service-identity integration still needs a key, and rotation is a separate practice from keeping the value out of the file.
- **Difficulty:** Medium
- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/secret-hygiene`
- **Revise:** `3_claude_code_tools_mcp.md` → Secret handling

Q49 — Answer: C

- **Why C is correct:** Packaging means separating engagement- and customer-specific values into documented, parameterized configuration with defaults and bundling the eval suite alongside the code, so a future team configures the asset instead of reading the whole implementation to work out what's safe to change. Do it while the build is fresh, because the knowledge of what's customer-specific is cheapest to write down before the people who hold it move to the next engagement.
- **Why not A:** This is the documented failure. The cost of not parameterizing doesn't show up until someone else tries to reuse the build — by which point the engineers who knew which values were load-bearing are gone.
- **Why not B:** The point of packaging is that a future team *doesn't* have to read the implementation. Documentation of which values are customer-specific matters more than a walkthrough of the code.
- **Why not D:** A bundled eval is one of the three pieces, but on its own it only confirms a guessed edit still works — it doesn't tell anyone which values are safe to change or supply the parameters to change them with.
- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.swe/packaging`
- **Revise:** `2_applications_and_integration.md` → Software Engineering Foundations (Packaging for reuse)

Q50 — Answer: A

- **Why A is correct:** Cache reads, cache writes, and regular input tokens are priced differently, so a naive "total input tokens" figure will misstate cost once caching is in play. Track them separately from the usage fields every response carries — the `usage` block on the raw Messages API, or `usage` and `total_cost_usd` on the Agent SDK's `ResultMessage`.
- **Why not B:** Output tokens are reported separately; there's no double-counting of output inside the input total.
- **Why not C:** Cached requests report cache-read tokens, not zero input — the reads are billed at a fraction of standard input rather than being invisible.
- **Why not D:** Cache reads cost roughly a tenth of standard input, not nothing, and a cache write costs more than base input — so input is very much still part of the model.
- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.cost/token-tracking`
- **Revise:** `4_model_selection_prompting_context.md` → Cost and Token Management

Section D — Score and Analysis

1. Domain breakdown

Total your correct answers per domain using the question numbers listed, then fill in the last three columns.

Domain	Questions	Your score	%	Official weight	Weighted contribution
Applications and Integration 1, 4, 7, 10, 13, 16, 19, 22, 25, 28, 31, 34, 37, 40, 43, 49	16	___ / 16	___%	33.1%	___
Model Selection and Optimization 2, 8, 15, 20, 26, 32, 38, 44, 50	9	___ / 9	___%	16.8%	___
Agents and Workflows 3, 11, 18, 27, 35, 42, 46	7	___ / 7	___%	14.7%	___
Prompt and Context Engineering 5, 14, 23, 30, 39, 47	6	___ / 6	___%	11.0%	___
Tools and MCP 6, 17, 29, 41, 48	5	___ / 5	___%	10.6%	___
Security and Safety 9, 21, 33, 45	4	___ / 4	___%	8.1%	___
Claude Code 12, 36	2	___ / 2	___%	3.1%	___
Eval, Testing, and Debugging 24	1	___ / 1	___%	2.6%	___
Total	50	___ / 50	___%	100%	___

2. Weighted readiness

Readiness = $\sum (\text{domain score \%} \times \text{official domain weight})$

Using official weights rather than the raw count means a miss in Applications and Integration costs you more than a miss in Eval — the same way the real exam prices it.

Worked example. Suppose you score 12/16 on Applications and Integration (75%), 7/9 on Model Selection (77.8%), 5/7 on Agents (71.4%), 5/6 on Prompt and Context (83.3%), 3/5 on Tools and MCP (60%), 4/4 on Security (100%), 2/2 on Claude Code (100%), and 0/1 on Eval (0%). That's 38/50 raw, or 76%. Weighted:

Domain	Score %	× Weight	Contribution
Applications and Integration	75.0%	33.1%	24.83
Model Selection and Optimization	77.8%	16.8%	13.07
Agents and Workflows	71.4%	14.7%	10.50
Prompt and Context Engineering	83.3%	11.0%	9.17
Tools and MCP	60.0%	10.6%	6.36
Security and Safety	100.0%	8.1%	8.10
Claude Code	100.0%	3.1%	3.10
Eval, Testing, and Debugging	0.0%	2.6%	0.00
Readiness			75.1%

Readiness of 75.1% against a raw 76% — close here, but the gap widens whenever your strong and weak domains sit on opposite sides of the weighting. Two more Applications and Integration misses traded for two Security wins would barely move the raw score and would cost you real readiness.

3. Readiness bands

Readiness	Reading	Next step
85%+	Ready to sit	Keep one weekly mock to stay warm; revise only flagged weak areas
75–84%	Nearly ready	Two focused sessions on your two weakest domains, then the next mock
65–74%	Real gaps	Re-run coaching sessions for every domain scoring under 70%, then the next mock
Below 65%	Not yet	Return to the source notes for the weakest domains before more mocks; mocks measure, they don't teach

The worked example above lands in "nearly ready": two focused sessions on Tools and MCP and on Agents and Workflows, then Mock Exam 3.

4. Weak-area capture

Do this before you close the exam, while the reasoning is still fresh:

1. For **every** question you missed, copy its **Tag** from Section C into `../progress_tracker.md`, along with the question number (`E2-Q29`) and one line on why you missed it — misread the stem, didn't know the fact, or talked yourself out of the right answer. The tag is what turns a wrong answer into a revision target; "I got question 29 wrong" is not actionable, `tools.mcp/transport-scope` is.
2. Also log any question you got **right but guessed on**. A coin flip that landed is not knowledge, and it will not land twice.
3. Any domain scoring **under 70% goes on the weak-areas list regardless of the overall total** — including Claude Code or Eval, where a single miss can put you under the line on a small question count. A high overall score does not retire a weak domain.
4. Work the tags, not the questions. Re-read the **Revise** pointer for each logged tag, then re-derive the answer from the notes rather than re-reading the explanation, which only teaches you to recognize this exam's wording.
5. Re-take a mock only after the logged weak areas have had a coaching session. Repeating a mock without revising in between measures your memory of the answer key, not your readiness.